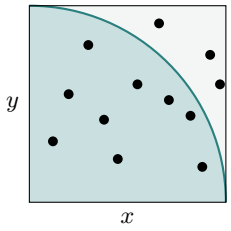
In-class exercise: Estimating π with threads

ITCS 4145, Parallel Programming, Fall 2026

Instructor: S M Ferdous · Date: Thursday, October 15, 2026 · Credit: Bonus

1 The method

Draw n points (x, y) with x and y uniform in $[0, 1)$. A point lies inside the quarter circle of radius 1 centered at the origin when $x^2 + y^2 < 1$. The quarter circle has area $\pi/4$ and the square has area 1, so the fraction of points inside approaches $\pi/4$ as n grows:



$$\pi \approx \frac{4 \cdot \text{hits}}{n}, \qquad \text{hits} = \text{number of points with } x^2 + y^2 < 1.$$

Each point is independent of every other, so the n points can be divided among threads, each thread counting the hits of its own points.

2 The starter file

`pi_start.c` (on Canvas) contains the timer, a random number generator, the counting loop, the sequential version, and `main`, which reads n and p from the command line and prints the estimate, the error, and the time. The generator holds its state in a structure; a thread that owns a `rng_t` variable owns its own sequence of numbers.

```
typedef struct { uint64_t state; } rng_t;
static void   rng_seed(rng_t *r, uint64_t seed);  // start a sequence
static double rng_unit(rng_t *r);                 // next number in [0, 1)
static long   count_hits(rng_t *r, long points);  // hits among points
```

The parallel version has two parts to complete, marked `TODO`: the thread function `work` and the function `pi_par` that creates, joins, and combines. The argument structure is given:

```
typedef struct {
    int      rank;    // which thread am I
    int      size;    // how many threads
    long     n;       // total number of points
    uint64_t seed;    // base seed; each thread adds its rank
    long    *hits;    // shared array of p slots; thread rank writes slot rank
} args_t;
```

Build with `gcc -O2 -Wall -pthread pi_start.c -o pi`. Run with `./pi n p`; `p = 0` runs the sequential version.

3 Steps

1. **Sequential.** Build and run the sequential version with $n = 10^7$, 10^8, and 10^9 on a Centaurus node:

   ```
   srun --partition=Centaurus --exclusive --cpus-per-task=36 ./pi 1000000000 0
   ```

   Record the estimate, the error, and the time for each n.

2. **First attempt: one shared counter.** Create a separate file named `pi_shared.c` and complete `work` and `pi_par` so that every thread adds its hits, one by one, into a single global variable `long hits`, and `pi_par` returns that variable after the joins. Divide the points into consecutive blocks,

n/p per thread, the remainder to the last thread, and seed thread `rank`'s generator with `seed + rank`. Run $n = 10^8$ with $p = 8$ three times and record the three estimates.

3. **Correction.** Create a file named `pi.c` and change `work` so that each thread counts in a local variable and writes the count once into `hits[rank]`, its own slot, and change `pi_par` so that it adds the p slots after the joins. Run $n = 10^8$ with $p = 8$ three times and record the three estimates. Run $p = 1$ and compare with the sequential result of step 1 at the same n.

4. **Timing.** With $n = 10^9$, run $p = 1, 2, 4, 8, 18, 36$. Record $T(p)$ and $S(p) = T_{\text{seq}}/T(p)$ with T_{seq} from step 1.

5. **Questions.** Answer in a few sentences each.

   (a) In step 2 the estimates are wrong, and may differ from run to run. Name the three machine steps behind `hits++` and state which memory location has more than one writer.

   (b) In step 3, $p = 1$ reproduces the sequential result, but $p = 8$ gives a different estimate at the same n. State why the result depends on p even though the program is correct.

4 Submission

Before the end of class, submit on Canvas your `pi.c` (the step 3 version) and one text or PDF file with the records of steps 1 to 4 and the answers of step 5.